# PeakAware

## Documentazione Tecnica

Ingegneria del Software · A.A. 2025/2026 · Gruppo 25

*Fabio Tomasi · Damiano Bolech · Giacomo Brugnara*

2026

# Indice

**1 Panoramica del Progetto** **2**

**2 Dipendenze** **2**
2.1 Produzione . . . . . 2
2.2 Sviluppo . . . . . 2

**3 Struttura delle Cartelle** **2**

**4 Endpoints API** **3**
4.1 Autenticazione — `/api/auth` . . . . . 3
4.2 Modulo Educazione — `/api/educazione` . . . . . 3

**5 Codici di Stato HTTP** **4**

**6 Autenticazione JWT** **4**

**7 Test Postman — Primo Commit** **5**

**8 Frontend Angular** **5**
8.1 Struttura delle cartelle . . . . . 5
8.2 Routes . . . . . 6
8.3 Funzionalità implementate . . . . . 6
8.4 Test funzionali . . . . . 6

**9 Pannello Admin — Gestione Quiz** **6**
9.1 Funzionalità admin implementate . . . . . 6
9.2 Route aggiuntive . . . . . 7
9.3 Gestione ruolo nel token JWT . . . . . 7
9.4 Test funzionali pannello admin . . . . . 7

# 1. Panoramica del Progetto

PeakAware è una piattaforma digitale (app + web) per avvicinare le persone alla montagna in modo consapevole, sicuro e sostenibile.
Il backend espone API REST consumate dal frontend Angular.

**Stack tecnologico:**

- Runtime: Node.js v24
- Framework HTTP: Express
- Database: MongoDB (Mongoose ODM)
- Autenticazione: JWT + bcryptjs
- Dev tool: nodemon

# 2. Dipendenze

## 2.1. Produzione

| Pacchetto | Versione | Ruolo |
|---|---|---|
| `express` | ^5.x | Framework HTTP: routing, middleware, REST |
| `mongoose` | ^8.x | ODM MongoDB: modelli, schema, validazioni |
| `dotenv` | ^16.x | Variabili d'ambiente da file `.env` |
| `bcryptjs` | ^2.x | Cifratura password |
| `jsonwebtoken` | ^9.x | Generazione e verifica token JWT |
| `cors` | ^2.x | Richieste cross-origin da Angular (porta 4200) |

## 2.2. Sviluppo

| Pacchetto | Versione | Ruolo |
|---|---|---|
| `nodemon` | ^3.x | Riavvio automatico del server ad ogni salvataggio |

# 3. Struttura delle Cartelle

| Percorso | Responsabilità |
|---|---|
| `backend/server.js` | Entry point: Express, middleware, routes |
| `backend/.env` | Variabili d'ambiente (non in Git) |

| | |
|---|---|
| `backend/src/config/database.js` | Connessione MongoDB tramite Mongoose |
| `backend/src/models/Utente.js` | Schema utente, cifratura password, verifica credenziali |
| `backend/src/models/Quiz.js` | Schema quiz con domande e risposte embedded |
| `backend/src/models/Video.js` | Schema video educativi e divulgativi |
| `backend/src/controllers/ /authController.js` | Logica registrazione e login |
| `backend/src/controllers/ /educazioneController.js` | CRUD quiz e video |
| `backend/src/routes/authRoutes.js` | Routing endpoint autenticazione |
| `backend/src/routes/ /educazioneRoutes.js` | Routing endpoint educazione |
| `backend/src/middleware/auth.js` | Verifica JWT e controllo ruolo admin |

# 4. Endpoints API

## 4.1. Autenticazione — `/api/auth`

| Metodo | URL | Protetto | Descrizione |
|---|---|---|---|
| POST | /api/auth/registrati | No | Crea account. Body: { email, password, eta } |
| POST | /api/auth/login | No | Autentica utente, restituisce token JWT |

## 4.2. Modulo Educazione — `/api/educazione`

| Metodo | URL | Protetto | Descrizione |
|---|---|---|---|
| GET | /api/educazione/quiz | No | Tutti i quiz. Filtri: ?categoria= &difficolta= |
| GET | /api/educazione/quiz/:id | No | Quiz completo con domande e risposte |
| POST | /api/educazione/quiz | Admin | Crea quiz (solo admin/-SAT) |
| PUT | /api/educazione/quiz/:id | Admin | Aggiorna quiz (solo admin/-SAT) |
| DELETE | /api/educazione/quiz/:id | Admin | Elimina quiz (solo admin/-SAT) |
| GET | /api/educazione/video | No | Tutti i video. Filtro: ?modulo= |

| | | | |
|---|---|---|---|
| POST | /api/educazione/video | Admin | Aggiunge video (solo admin/SAT) |

## 5. Codici di Stato HTTP

Ogni risposta include sempre `successo` (boolean) e `messaggio` (stringa).
In caso di errore è presente anche `errore` con il dettaglio tecnico.

| Codice | Nome | Quando viene restituito |
|---|---|---|
| **200** | OK | Operazione completata (GET, PUT, DELETE riusciti) |
| **201** | Created | Risorsa creata (POST /registrati, POST /quiz, POST /video) |
| **400** | Bad Request | Dati mancanti o non validi nel body |
| **401** | Unauthorized | Token assente, non valido o scaduto; credenziali errate |
| **403** | Forbidden | Utente autenticato ma senza permessi admin |
| **404** | Not Found | Risorsa non trovata (es. ID quiz inesistente) |
| **500** | Internal Server Error | Errore imprevisto o timeout database |

**Esempio risposta successo (201):**

```
{ "successo": true, "messaggio": "Registrazione avvenuta con
   successo",
  "dati": { "id": "...", "email": "...", "ruolo": "utente", "
     token": "..." } }
```

**Esempio risposta errore (401):**

```
{ "successo": false, "messaggio": "Accesso negato: token
   mancante" }
```

## 6. Autenticazione JWT

Il sistema usa JSON Web Token per gestire le sessioni stateless.
Il token viene generato al login, inviato al frontend e allegato ad ogni richiesta successiva nell'header `Authorization: Bearer <token>`.
Il middleware `proteggi` lo verifica e aggiunge l'utente a `req.utente`.
Il token scade dopo 7 giorni.

# 7. Test Postman — Primo Commit

| # | Metodo | URL | Risultato atteso | Esito |
|---|---|---|---|---|
| 1 | GET | / | Server attivo | **PASS** |
| 2 | POST | /api/auth/registrati | 201 + token JWT | **PASS** |
| 3 | POST | /api/auth/login | 200 + token JWT | **PASS** |
| 4 | POST | /api/educazione/quiz (no token) | 401 token mancante | **PASS** |
| 5 | POST | /api/educazione/quiz (utente) | 403 permessi insufficienti | **PASS** |

Tutti i test superati. Il backend core è funzionante.

# 8. Frontend Angular

Il frontend è sviluppato con Angular 21 in modalità standalone. Comunica con il backend esclusivamente tramite chiamate HTTP a `localhost:3000`.

**Stack frontend:**

- Framework: Angular 21
- Linguaggio: TypeScript
- Stile: CSS puro per componente
- Comunicazione backend: `HttpClient` con `provideHttpClient()`

## 8.1. Struttura delle cartelle

| Percorso | Responsabilità |
|---|---|
| `frontend/src/app/app.routes.ts` | Routing dell'applicazione |
| `frontend/src/app/app.config.ts` | Configurazione globale: router e HttpClient |
| `frontend/src/app/core/services/ /auth.service.ts` | Chiamate HTTP a /api/auth, gestione token JWT |
| `frontend/src/app/features/auth/ /login/` | Componente login |
| `frontend/src/app/features/auth/ /registrazione/` | Componente registrazione |
| `frontend/src/app/features/ /education/quiz-list/` | Componente lista quiz con filtro per categoria |

## 8.2. Routes

| Path | Comportamento |
|---|---|
| `/` | Redirect automatico a /login |
| `/login` | Schermata di accesso |
| `/registrazione` | Schermata di creazione account |
| `/quiz` | Lista quiz — richiede autenticazione |
| `/**` | Qualsiasi path sconosciuto → redirect a /login |

## 8.3. Funzionalità implementate

- Registrazione utente con salvataggio token JWT e redirect automatico
- Login con verifica credenziali e redirect alla lista quiz
- Protezione route: se non autenticato, redirect al login
- Lista quiz con filtro per categoria
- Logout con rimozione token e redirect al login
- Gestione errori con messaggi all'utente

## 8.4. Test funzionali

| # | Azione | Risultato atteso | Esito |
|---|---|---|---|
| 1 | Registrazione nuovo utente | Redirect a /quiz + token salvato | **PASS** |
| 2 | Login utente esistente | Redirect a /quiz + token salvato | **PASS** |
| 3 | Accesso /quiz senza token | Redirect a /login | **PASS** |
| 4 | Lista quiz vuota | Messaggio nessun quiz disponibile | **PASS** |
| 5 | Logout | Token rimosso, redirect a /login | **PASS** |

# 9. Pannello Admin — Gestione Quiz

Gli utenti con ruolo `admin` hanno accesso a funzionalità aggiuntive per la gestione dei contenuti educativi, corrispondenti al componente Gestione Contenuti del D2 sezione 1.3.

## 9.1. Funzionalità admin implementate

- Card + in cima alla griglia quiz per creare un nuovo quiz
- Pulsante modifica () su ogni card per aggiornare un quiz esistente
- Pulsante elimina () su ogni card con conferma prima dell'eliminazione
- Form dedicato su `/admin/quiz-form` per creazione e modifica
- Il form gestisce domande e risposte con aggiunta/rimozione dinamica

## 9.2. Route aggiuntive

| Path | Comportamento |
|---|---|
| `/admin/quiz-form` | Form creazione nuovo quiz (solo admin) |
| `/admin/quiz-form/:id` | Form modifica quiz esistente (solo admin) |

## 9.3. Gestione ruolo nel token JWT

Il token JWT include ora il campo `ruolo` nel payload, oltre all'id utente. Il frontend decodifica il payload in base64 senza librerie esterne per determinare i permessi e mostrare o nascondere i controlli admin.

## 9.4. Test funzionali pannello admin

| # | Azione | Risultato atteso | Esito |
|---|---|---|---|
| 1 | Login come admin | Card + visibile in griglia | **PASS** |
| 2 | Click card + | Redirect a /admin/quiz-form | **PASS** |
| 3 | Creazione nuovo quiz | Quiz salvato, redirect a /quiz | **PASS** |
| 4 | Click modifica su card | Form precompilato con dati quiz | **PASS** |
| 5 | Salvataggio modifica | Quiz aggiornato, redirect a /quiz | **PASS** |
| 6 | Click elimina con conferma | Quiz rimosso dalla lista | **PASS** |
| 7 | Login come utente normale | Card + e controlli non visibili | **PASS** |